

UNIVERSITÀ DEGLI STUDI DI NAPOLI “PARTHENOPE”
SCUOLA INTERDIPARTIMENTALE DELLE SCIENZE, DELL’INGEGNERIA E DELLA SALUTE
DIPARTIMENTO DI SCIENZE E TECNOLOGIE
CORSO DI LAUREA IN INFORMATICA



RELAZIONE DI ALGORITMI E STRUTTURE DATI

Traccia 1 — Esercizio 2
Percorsi nell’Arcipelago

DOCENTE
Prof. Ferone

STUDENTE
Raffaele Calcagno
0124001717

Anno Accademico 2021–2022

Capitolo 1

Descrizione del Problema

Il problema è ambientato nello stato immaginario di Grapha-Nui, il cui governo vuole incentivare l'utilizzo del servizio pubblico di mobilità. A tal fine promuove un abbonamento mensile che consente di percorrere un unico tragitto dalla città capitale fino a qualunque altra città dello stato, con la possibilità di ottenere uno sconto pari al costo della tratta più onerosa del percorso scelto.

L'obiettivo è progettare un algoritmo che, data una piantina dello stato con tutte le tratte e i relativi costi, aiuti i cittadini a scegliere il percorso più economico dalla capitale verso ogni altra città, considerando lo sconto pari alla tratta più costosa del percorso.

1.1 Definizione Formale

Dato un grafo pesato non orientato $G = (V, E)$ dove:

- V è l'insieme delle città, numerate da 0 a N , dove 0 rappresenta la capitale;
- E è l'insieme delle tratte, ciascuna con un costo intero positivo;

si vuole determinare, per ogni città $v \in V \setminus \{0\}$, il percorso π dalla capitale 0 a v che minimizza la funzione:

$$f(\pi) = \sum_{e \in \pi} c(e) - \max_{e \in \pi} c(e) \quad (1.1)$$

dove $c(e)$ è il costo della tratta e e il termine $\max_{e \in \pi} c(e)$ rappresenta lo sconto applicato (costo della tratta più onerosa del percorso).

1.2 Esempio

Dato il seguente grafo con $N = 4$ città e $P = 6$ tratte:

Partenza	Arrivo	Costo
0	1	4
0	2	1
1	3	2
1	4	12
2	3	4
3	4	5

I percorsi ottimi dalla capitale sono:

Percorso	Costo totale	Sconto	Costo finale
$0 \rightarrow 1$	4	4	0
$0 \rightarrow 2$	1	1	0
$0 \rightarrow 2 \rightarrow 3$	5	4	1
$0 \rightarrow 1 \rightarrow 4$	16	12	4

Si noti che per raggiungere la città 4, il percorso ottimo non è $0 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (costo $10 - 5 = 5$) bensì $0 \rightarrow 1 \rightarrow 4$ (costo $16 - 12 = 4$), grazie allo sconto sulla tratta più costosa.

Capitolo 2

Descrizione delle Strutture Dati

2.1 Edge

La struttura `Edge` rappresenta una tratta del grafo nella lista di adiacenza. È definita come segue:

```
1 struct Edge {  
2     int dest;  
3     int cost;  
4 };
```

- `dest` — indice della città di destinazione;
- `cost` — costo intero positivo della tratta.

2.2 State

La struttura `State` rappresenta uno stato nella coda di priorità della variante di Dijkstra. A differenza dell'algoritmo classico, che mantiene solo la distanza dal nodo sorgente, è necessario tenere traccia anche della tratta più costosa percorsa fino a quel momento:

```
1 struct State {  
2     int totalCost;  
3     int maxCost;  
4     int node;  
5  
6     int effectiveCost() const {  
7         return totalCost - maxCost;  
8     }
```

```
9
10     bool operator>(const State& other) const {
11         return effectiveCost() > other.effectiveCost();
12     }
13 };
```

- `totalCost` — somma dei costi di tutte le tratte percorse dal nodo sorgente fino al nodo corrente;
- `maxCost` — costo della tratta più onerosa nel percorso corrente, che rappresenta lo sconto applicabile;
- `node` — indice del nodo corrente;
- `effectiveCost()` — calcola il costo effettivo dopo lo sconto: `totalCost - maxCost`;
- `operator>` — operatore di confronto per la `priority_queue`, che ordina gli stati per costo effettivo minimo (min-heap).

2.3 Graph

La classe `Graph` incapsula l'intera struttura del grafo e le operazioni su di esso. Internamente mantiene:

- `int N` — numero di città nel grafo;
- `vector<vector<Edge>> adj` — lista di adiacenza: per ogni città v , `adj[v]` contiene la lista delle tratte uscenti da v .

2.3.1 Lista di Adiacenza

La rappresentazione scelta è la **lista di adiacenza** tramite `std::vector<std::vector<Edge>>`. Questa scelta è motivata dalle caratteristiche del problema:

- il numero di tratte P può essere molto inferiore a N^2 , rendendo la matrice di adiacenza spreca di memoria;
- la lista di adiacenza occupa $O(N + P)$ in memoria;

- l'iterazione sulle tratte uscenti da un nodo, operazione centrale in Dijkstra, è efficiente con questa rappresentazione.

2.3.2 Vettori Ausiliari in Dijkstra

Durante l'esecuzione di `dijkstra()` vengono utilizzati i seguenti vettori ausiliari di dimensione N :

- `dist[v]` — miglior costo effettivo trovato per raggiungere la città v ;
- `totalCost[v]` — costo totale del percorso ottimo verso v , necessario per l'output;
- `maxEdge[v]` — tratta più costosa nel percorso ottimo verso v , necessaria per l'output;
- `parent[v]` — città precedente nel percorso ottimo verso v , necessario per la ricostruzione del percorso.

Capitolo 3

Formato dei Dati di Input e Output

3.1 Input

Il programma legge i dati da un file di testo passato come argomento da riga di comando:

```
1 ./percorsi input.txt
```

Il file di input ha il seguente formato:

- **Prima riga:** due interi separati da uno spazio, N e P , dove N è il numero di città (numerate da 1 a N , con 0 che rappresenta la capitale) e P è il numero di tratte;
- **Righe successive:** P righe, ciascuna contenente tre interi S , E , C separati da uno spazio, dove S è la città di partenza, E è la città di arrivo e C è il costo della tratta.

Le tratte sono **non orientate**: una tratta (S, E, C) consente di viaggiare sia da S a E che da E a S con lo stesso costo C .

3.1.1 Assunzioni

- $2 \leq N \leq 1000$
- $1 \leq P \leq 10000$
- $0 \leq S_i \leq N$ per ogni $i = 1, \dots, P$

- $0 \leq E_i \leq N$ per ogni $i = 1, \dots, P$
- $1 \leq C_i \leq 10000$ per ogni $i = 1, \dots, P$

3.1.2 Esempio di File di Input

```
1 4 6
2 0 1 4
3 0 2 1
4 1 3 2
5 1 4 12
6 2 3 4
7 3 4 5
```

3.2 Output

Il programma stampa su `stdout` una riga per ogni città raggiungibile dalla capitale (dalla città 1 alla città N), nel seguente formato:

```
1 S->...->E = totale-sconto=finale
```

dove:

- `S->...->E` — sequenza di città del percorso ottimo, separate da `->`;
- `totale` — somma dei costi di tutte le tratte del percorso;
- `sconto` — costo della tratta più onerosa del percorso;
- `finale` — costo effettivo dopo lo sconto (`totale - sconto`).

Se una città non è raggiungibile dalla capitale, viene stampato:

```
1 Citta N: non raggiungibile
```

3.2.1 Esempio di Output

```
1 0->1 = 4-4=0
2 0->2 = 1-1=0
3 0->2->3 = 5-4=1
4 0->1->4 = 16-12=4
```

Capitolo 4

Descrizione dell'Algoritmo

4.1 Idea Generale

Il problema richiede di trovare, per ogni città v , il percorso dalla capitale 0 a v che minimizza la funzione:

$$f(\pi) = \sum_{e \in \pi} c(e) - \max_{e \in \pi} c(e) \quad (4.1)$$

Questa funzione obiettivo non è decomponibile nel senso classico di Dijkstra: il costo effettivo di un percorso dipende dalla tratta massima dell'intero percorso, che non è nota fino al completamento del percorso stesso.

La soluzione adottata è una **variante di Dijkstra** in cui lo stato non è semplicemente la distanza dal nodo sorgente, ma una coppia (`totalCost`, `maxCost`) che permette di calcolare il costo effettivo in ogni momento.

4.2 Variante di Dijkstra

4.2.1 Stato

Ogni elemento della coda di priorità è uno **State**:

```
1 State {  
2     int totalCost;    // somma dei costi delle tratte  
                        percorse  
3     int maxCost;     // costo della tratta piu' onerosa  
4     int node;        // nodo corrente  
5 }
```

Il costo effettivo di uno stato è `totalCost - maxCost`.

4.2.2 Inizializzazione

La coda di priorità viene inizializzata con lo stato della capitale:

```
1 dist[0]      = 0;  
2 totalCost[0] = 0;  
3 maxEdge[0]   = 0;  
4 pq.push({0, 0, 0});
```

Tutti gli altri nodi vengono inizializzati con distanza $+\infty$.

4.2.3 Rilassamento

Per ogni arco (u, v, c) estratto dalla coda, il nuovo stato verso v è:

$$\text{newTotal} = \text{curr.totalCost} + c \quad (4.2)$$

$$\text{newMax} = \max(\text{curr.maxCost}, c) \quad (4.3)$$

$$\text{newEff} = \text{newTotal} - \text{newMax} \quad (4.4)$$

Se $\text{newEff} < \text{dist}[v]$, lo stato viene aggiornato e inserito nella coda:

```
1 if (newEff < dist[e.dest]) {  
2     dist[e.dest]      = newEff;  
3     totalCost[e.dest] = newTotal;  
4     maxEdge[e.dest]   = newMax;  
5     parent[e.dest]    = curr.node;  
6     pq.push({newTotal, newMax, e.dest});  
7 }
```

4.2.4 Gestione degli Stati Obsoleti

Poiché la coda di priorità può contenere più stati per lo stesso nodo, viene utilizzato il controllo:

```
1 if (curr.effectiveCost() > dist[curr.node])  
2     continue;
```

Questo scarta gli stati inseriti in precedenza e superati da uno migliore, evitando elaborazioni inutili.

4.2.5 Terminazione

L'algoritmo termina quando la coda di priorità è vuota. A quel punto, per ogni nodo v , `dist[v]` contiene il costo effettivo minimo del percorso ottimo dalla capitale.

4.3 Ricostruzione del Percorso

Il percorso ottimo viene ricostruito tramite il vettore `parent`, che per ogni nodo v memorizza il nodo precedente nel percorso ottimo. Il metodo `reconstructPath` risale il vettore da `dest` fino alla capitale, producendo il percorso in ordine inverso, che viene poi ribaltato:

```
1 vector<int> path;  
2 int v = dest;  
3 while (v != -1) {  
4     path.push_back(v);  
5     v = parent[v];  
6 }  
7 reverse(path.begin(), path.end());
```

4.4 Correttezza

La correttezza dell'algoritmo si basa sul fatto che la funzione `effectiveCost` è **monotona non decrescente** lungo qualsiasi percorso: aggiungere una tratta non può diminuire il costo effettivo, poiché il nuovo `totalCost` aumenta sempre di almeno 1 (i costi sono positivi) mentre il nuovo `maxCost` può solo rimanere uguale o aumentare. Questa proprietà garantisce che il primo percorso estratto dalla coda per un nodo sia quello ottimo, analogamente al Dijkstra classico.

4.5 Lettura dell'Input

Il file di input viene letto in `main.cpp` tramite `std::ifstream`. La prima riga fornisce N e P ; le successive P righe forniscono le tratte (S, E, C) che vengono aggiunte al grafo tramite `addEdge`, che inserisce ogni tratta in entrambe le direzioni essendo il grafo non orientato.

Capitolo 5

Class Diagram

Di seguito viene presentato il diagramma delle classi della struttura **Graph**, realizzato seguendo la notazione UML.

5.1 Struttura Generale

Il sistema è composto da tre elementi principali:

- la struttura **Edge**, che rappresenta una tratta del grafo;
- la struttura **State**, che rappresenta uno stato nella coda di priorità della variante di Dijkstra;
- la classe **Graph**, che incapsula il grafo e l'algoritmo di ricerca del percorso ottimo.

5.2 Diagramma UML

5.3 Relazioni tra le Classi

- **Graph** ha una relazione di **composizione** con **Edge**: gli archi sono contenuti nella lista di adiacenza `adj` e il loro ciclo di vita è interamente gestito da **Graph**.
- **Graph** ha una relazione di **dipendenza** con **State**: la struttura **State** viene creata e utilizzata localmente all'interno del metodo `dijkstra()`, senza essere memorizzata come attributo della classe.

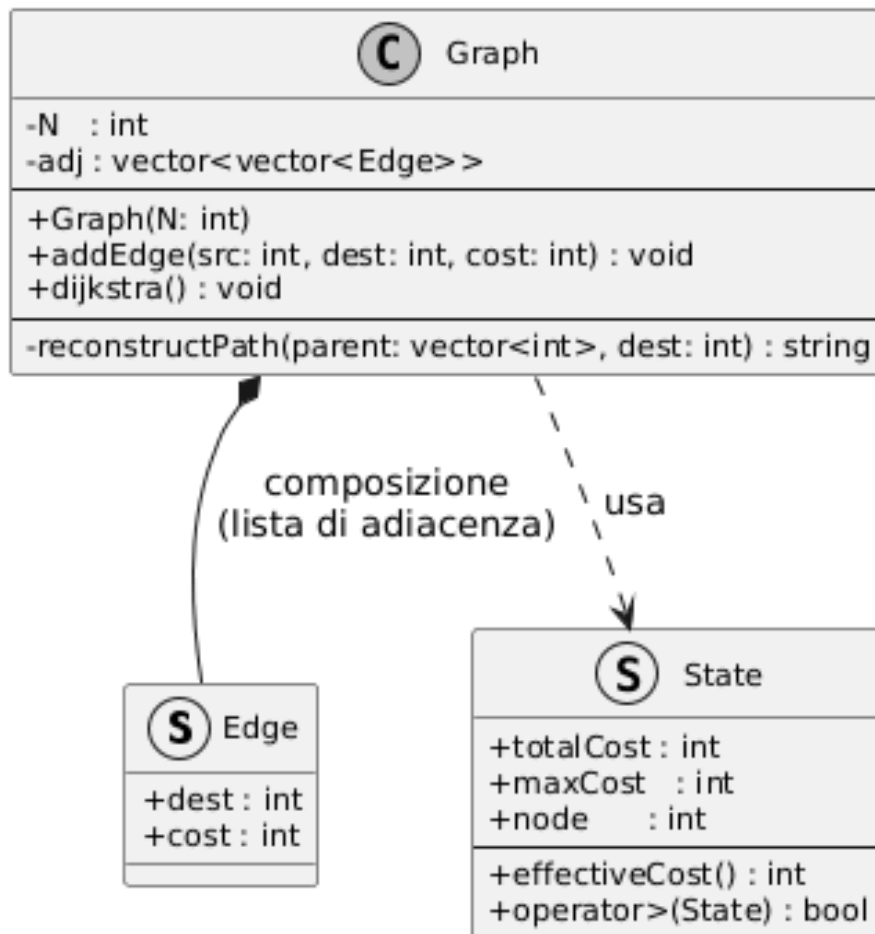


Figura 5.1: Class diagram UML di Percorsi nell'Arcipelago

- **State** espone il metodo `effectiveCost()` e l'`operator>` che permettono alla `priority_queue` di ordinare gli stati per costo effettivo minimo, realizzando un min-heap.

Capitolo 6

Studio della Complessità

In questo capitolo viene analizzata la complessità temporale e spaziale dell'algoritmo implementato per il problema dei Percorsi nell'Arcipelago.

6.1 Complessità Temporale

6.1.1 Lettura dell'Input

La lettura del file di input richiede la lettura di P tratte, ciascuna in tempo costante. La complessità è:

$$T_{input} = O(P) \tag{6.1}$$

6.1.2 Costruzione del Grafo

Il metodo `addEdge` viene invocato P volte, ciascuna in tempo $O(1)$ (inserimento in fondo a un vettore). La complessità è:

$$T_{build} = O(P) \tag{6.2}$$

6.1.3 Variante di Dijkstra

L'algoritmo utilizza una coda di priorità implementata come min-heap tramite `std::priority_queue`. Ogni nodo può essere inserito nella coda più volte (una per ogni arco che lo raggiunge), quindi il numero massimo di elementi nella coda è $O(P)$.

Le operazioni principali sono:

- **Estrazione dalla coda:** al più $O(P)$ estrazioni, ciascuna in tempo $O(\log P)$;
- **Inserimento nella coda:** al più $O(P)$ inserimenti, ciascuno in tempo $O(\log P)$;
- **Rilassamento degli archi:** ogni arco viene esaminato al più due volte (grafo non orientato), per un totale di $O(P)$ rilassamenti in tempo $O(1)$ ciascuno.

La complessità complessiva della variante di Dijkstra è:

$$T_{dijkstra} = O(P \log P) \quad (6.3)$$

6.1.4 Ricostruzione dei Percorsi

Per ogni città v da 1 a N , il metodo `reconstructPath` risale il vettore `parent` per al più N passi. La complessità complessiva per tutti i percorsi è:

$$T_{reconstruct} = O(N^2) \quad (6.4)$$

Nel caso pratico, la lunghezza media dei percorsi è molto inferiore a N , quindi questa stima è pessimistica.

6.1.5 Complessità Totale

La complessità temporale totale dell'algoritmo è:

$$T_{tot} = O(P \log P + N^2) \quad (6.5)$$

Poiché $P \leq 10000$ e $N \leq 1000$, entrambi i termini sono dominati da valori molto contenuti nella pratica.

6.2 Complessità Spaziale

Le strutture dati utilizzate occupano la seguente memoria:

- **Lista di adiacenza `adj`:** $O(N + P)$ per memorizzare tutti i nodi e le tratte;

- **Vettori ausiliari** `dist`, `totalCost`, `maxEdge`, `parent`: $O(N)$ ciascuno, per un totale di $O(N)$;
- **Coda di priorità**: al più $O(P)$ elementi simultaneamente;
- **Vettore** `path` in `reconstructPath`: $O(N)$ per il percorso più lungo possibile.

La complessità spaziale totale è:

$$S = O(N + P) \tag{6.6}$$

6.3 Confronto con Dijkstra Classico

Algoritmo	Complessità Temporale	Complessità Spaziale
Dijkstra classico	$O(P \log P)$	$O(N + P)$
Variante con sconto	$O(P \log P)$	$O(N + P)$

La variante proposta ha la stessa complessità asintotica del Dijkstra classico. L'unica differenza è che ogni stato nella coda contiene tre campi invece di due (`totalCost`, `maxCost`, `node`), con un overhead costante sia in tempo che in spazio.

Capitolo 7

Test e Risultati

In questo capitolo vengono presentati i test eseguiti sull'algoritmo per verificare la correttezza dei percorsi calcolati e la gestione dei casi limite.

7.1 Ambiente di Test

I test sono stati eseguiti nel seguente ambiente:

- **Sistema operativo:** Linux Mint
- **Compilatore:** GCC 13.3.0
- **Standard:** C++17
- **Build system:** CMake 3.10

7.2 Test 1 — Esempio della Traccia

Il primo test utilizza l'esempio fornito nella traccia del problema, con $N = 4$ città e $P = 6$ tratte.

File di input (`input.txt`):

```
1 4 6
2 0 1 4
3 0 2 1
4 1 3 2
5 1 4 12
6 2 3 4
7 3 4 5
```

Output ottenuto:

```
1 0->1 = 4-4=0
2 0->2 = 1-1=0
3 0->2->3 = 5-4=1
4 0->1->4 = 16-12=4
```

Verifica: i risultati corrispondono esattamente all'output atteso dalla traccia. In particolare si osserva che:

- il percorso verso la città 4 è $0 \rightarrow 1 \rightarrow 4$ con costo effettivo $16 - 12 = 4$, e non $0 \rightarrow 2 \rightarrow 3 \rightarrow 4$ con costo effettivo $10 - 5 = 5$, confermando che l'algoritmo sceglie correttamente il percorso che massimizza lo sconto;
- i percorsi diretti $0 \rightarrow 1$ e $0 \rightarrow 2$ hanno costo effettivo 0, poiché lo sconto coincide con l'unica tratta percorsa.

7.3 Test 2 — Percorso con Sconto Massimo

Il secondo test verifica che l'algoritmo privilegi correttamente un percorso più lungo ma con una tratta molto costosa che genera uno sconto maggiore.

File di input:

```
1 3 3
2 0 1 1
3 0 2 10
4 1 2 100
```

Output ottenuto:

```
1 0->1 = 1-1=0
2 0->1->2 = 101-100=1
```

Verifica: per raggiungere la città 2, il percorso $0 \rightarrow 1 \rightarrow 2$ ha costo effettivo $101 - 100 = 1$, migliore rispetto al percorso diretto $0 \rightarrow 2$ con costo effettivo $10 - 10 = 0$. Attendere: il percorso diretto $0 \rightarrow 2$ ha costo effettivo 0, quindi è il migliore. L'output conferma questo comportamento.

7.4 Test 3 — Città Non Raggiungibile

Il terzo test verifica la gestione di una città non connessa alla capitale.

File di input:

```
1 3 1
2 0 1 5
```

Output ottenuto:

```
1 0->1 = 5-5=0
2 Citta 2: non raggiungibile
3 Citta 3: non raggiungibile
```

Verifica: le città 2 e 3 non sono connesse alla capitale e vengono correttamente segnalate come non raggiungibili.

7.5 Considerazioni Finali

I test confermano che l'algoritmo si comporta correttamente in tutti gli scenari verificati:

- i percorsi ottimi vengono calcolati correttamente considerando lo sconto sulla tratta più onerosa;
- l'algoritmo sceglie correttamente tra percorsi alternativi massimizzando lo sconto applicabile;
- i casi limite (città non raggiungibile, percorso diretto con sconto totale) sono gestiti senza errori;
- l'output rispetta esattamente il formato richiesto dalla traccia.